Assignment 3: Quantum Machine Learning

ECE484 - Quantum Algorithms and Software

Pen-and-paper + Qiskit + Qiskit Machine Learning + sQUlearn

1. Learning objectives

By completing this assignment, students will then be able to:

1. Formulate a constrained classical optimization problem as a QUBO and map it to an Ising Hamiltonian.
2. Use Qiskit QAOA helpers to approximately solve the resulting combinatorial problem.
3. Derive and implement single-qubit encoders that match target expectation distributions on X, Y, Z.
4. Build and train a simple variational quantum classifier from scratch using Qiskit and Qiskit Machine Learning.
5. Use sQUlearn abstractions for multiclass quantum kernel methods and multi-output QNN regression.

2. Environment setup

This assignment uses two separate pixi environments: `qiskit` for Q1–Q3 (Qiskit 2.x + Qiskit Machine Learning) and `squlearn` for Q4 (sQUlearn stack).

Inside the assignment root: run `pixi install -e qiskit` and `pixi install -e squlearn`.

3. Testing commands

To run tests of your implementations, use the matching environment for each part:

```
# For Q1--Q3
pixi run -e qiskit tests_qiskit
# For Q4
pixi run -e squlearn tests_squlearn
```

Run a single question with:

```
pixi run -e qiskit tests_q1
pixi run -e qiskit tests_q2
pixi run -e qiskit tests_q3
pixi run -e squlearn tests_q4
```

Run the longer Q3 re-uploading sweep separately with:

```
pixi run -e qiskit tests_q3_reuploading_sweep
```

Run one specific test with `pytest` in the right environment, e.g.:

```
pixi run -e qiskit pytest -v -s tests/test_Q2.py::
   test_noisy_simulation_on_bell_family_generates_comparison_plot
```

Test outputs. Some tests will save images in `output/`. You can open these files to visually to better interpret the results of your implementations.

4. Deliverables

1. `paper/answers.pdf` for all pen-and-paper parts (Q1–Q4 written discussion parts; please number your answers correctly according to the questions).
2. Completed coding TODOs in `src/Q1_qubo_qaoa.py`, `src/Q2_distribution_matching.py`, `src/Q3_vqc_from_scratch.py`, and `src/Q4_squlearn_models.py`.

That should lead to passing all tests when running `pixi run -e qiskit tests_qiskit` and `pixi run -e squlearn tests_squlearn`. If you are not passing a test and believe your implementation is correct, please state this in your written submission and explain why.

5. Questions

Q1 - QUBO modeling and QAOA implementation (25 pts)

Goal. In this question, you will formulate a conflict-constrained optimization problem as a QUBO, convert it to an Ising Hamiltonian, and solve the assigned six-item instance with a small QAOA experiment. You will compare the variational result against the exact optimum to assess the quality of the approximation.

Problem context. Consider a conflict-constrained selection problem with n candidate items indexed by $i \in \{0, \ldots, n-1\}$. Item i has value $v_i > 0$. The conflict set is

$$\mathcal{E} \subseteq \{(i, j) \mid 0 \leq i < j \leq n-1\},$$

and each edge $(i, j) \in \mathcal{E}$ means that items i and j are incompatible, so they cannot both be selected.

Let $x_i \in \{0, 1\}$ denote whether item i is selected. The general model uses one binary variable, hence one qubit, per item. Therefore, an instance with n items uses n qubits. In this assignment, you must *derive the formulation for general* n, but the coding and numerical parts must use the fixed six-item instance given below so that the problem fits exactly within the six-qubit budget.

We will use the following six-item problem for Q1.c:

$$(v_0, v_1, v_2, v_3, v_4, v_5) = (4, 5, 3, 6, 2, 5),$$

with

$$\mathcal{E} = \{(0, 1), (1, 3), (2, 3), (2, 4), (4, 5)\}.$$

Hardware/software constraints. Use exactly this 6-item instance so your implementation fits within 6 qubits or less, use QAOA depth $p = 1$ for your main experiment, and use local noiseless simulation with qiskit.

Q1.a (paper). Derive a QUBO objective for this problem, justify the penalty scale, and map it to an Ising Hamiltonian.

Q1.b (coding). Implement `build_profit_conflict_qubo()`, `qubo_energy()`, `qubo_to_ising_hamiltonian()`, and `solve_with_qaoa_grid()` in `src/Q1_qubo_qaoa.py`. The Q1 tests will write, in `output/`, at least `Q1_b_qaoa_ansatz_circuit_test.png`.

Q1.c (paper). The tests will run your QAOA solver on six-item instance above and write `output/Q1_c_qaoa_probability_energy_histogram.png` using the final basis-state probabilities and their corresponding QUBO energies, discuss the result in this figure and how the classical post-processing helps.

Q2 - Structured encoder design from simple gate blocks (25 pts)

Goal. In this question, you will build a few specific circuit families whose behavior is easy to interpret directly from their gate structure. Reference SVG and GIF files are provided in `references/` before the question to help you understand what each family should do as x varies.

Q2.a (paper). Start with two one-qubit families that both stay on the same XZ-plane half-circle, but do not move along that path in the same way.

Family A: basic XZ-plane sweep

$$|\psi_A(x)\rangle = R_Y(\pi x)|0\rangle.$$

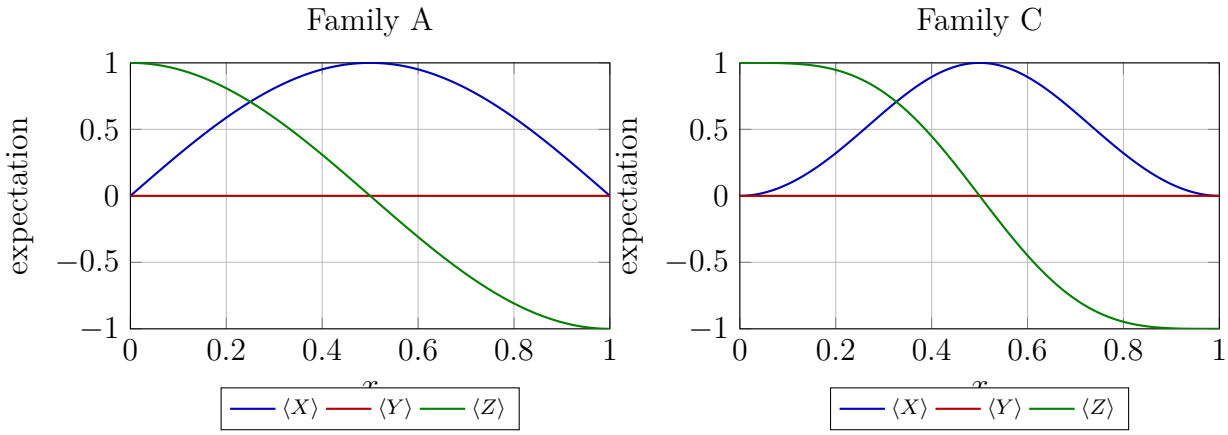
Reference files: see `Q2_b_xz_half_circle_expectations.png` and `Q2_b_xz_half_circle_block.gif`.

Family C: remapped XZ-plane sweep

$$u(x) = 3x^2 - 2x^3, \qquad |\psi_C(x)\rangle = R_Y(\pi u(x))|0\rangle.$$

Reference files: see `Q2_b_mapped_xz_half_circle_expectations.png` and `Q2_b_mapped_xz_half_circle_bloch.gif`.

In your written answer, give the Bloch coordinates $(\langle X\rangle, \langle Y\rangle, \langle Z\rangle)$ for Family A, the same coordinates for Family C, and a short explanation of what the mapping $u(x)$ changes and what it does not change.



Q2.b (coding + short explanation). Here are two additional two one-qubit targets:

Family B: XY-plane phase sweep

$$|\psi_A(x)\rangle = R_Z(2\pi x)H|0\rangle, \qquad \mathbf{m}_B(x) = \big(\cos(2\pi x),\, \sin(2\pi x),\, 0\big),$$

with reference files `Q2_b_xy_plane_circle_expectations.png` and `Q2_b_xy_plane_circle_bloch.gif`.

Family D: re-uploaded phase family

$$|\psi_D(x)\rangle = R_Z(2\pi x)R_X(2\pi x)|0\rangle,$$

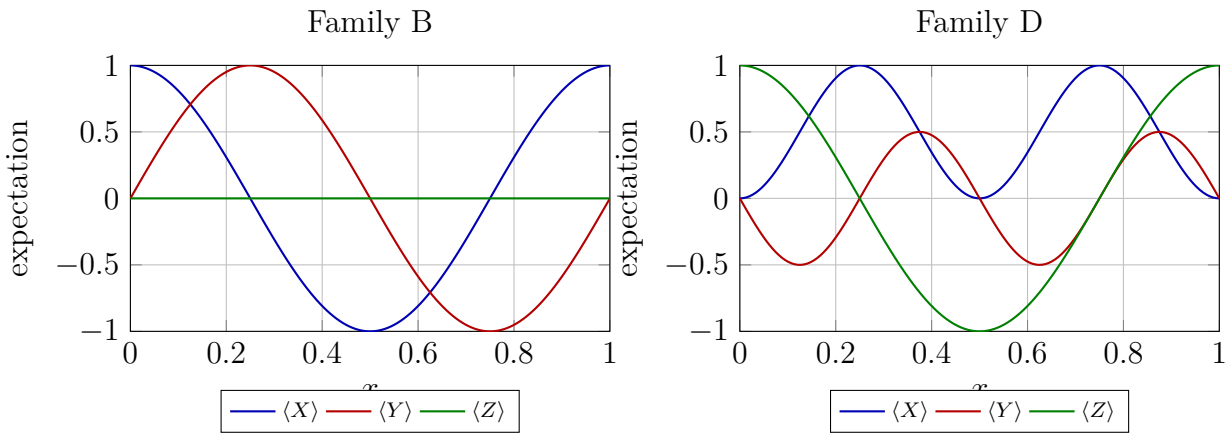
whose Bloch coordinates are

$$\mathbf{m}_D(x) = \big(\sin^2(2\pi x),\, -\sin(2\pi x)\cos(2\pi x),\, \cos(2\pi x)\big).$$

Reference files: see `Q2_b_reuploaded_phase_expectations.png` and `Q2_b_reuploaded_phase_bloch.gif`.

Implement `pauli_expectations_from_circuit()`, `build_xz_half_circle_encoder()`, `build_mapped_xz_half_circle_encoder()`, `build_xy_plane_circle_encoder()`, and `build_reuploaded_phase_encoder()` in `src/Q2_distribution_matching.py`.

In 2–4 sentences, explain why Family C is an example of a classical input mapping and why Family D is an example of data re-uploading.



Q2.c (paper + coding). Now move to the two-qubit Bell-like family. Starting from $|00\rangle$, apply

$$R_Y(\pi x) \text{ on qubit } 0, \qquad \text{then} \qquad CX(0,1).$$

This produces

$$|\Psi(x)\rangle = \cos\left(\frac{\pi x}{2}\right)|00\rangle + \sin\left(\frac{\pi x}{2}\right)|11\rangle.$$

Study this family in the computational basis ordered as $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ and in the X basis ordered as $|++\rangle, |+-\rangle, |-+\rangle, |--\rangle$.

Reference files: `Q2_c_bell_family_z_basis_probabilities.png`, `Q2_c_bell_family_x_basis_probabilities.png`, `Q2_c_bell_family_qsphere.gif`.

Coding. implement `build_bell_family_circuit()` and `basis_probabilities_from_circuit()`.

Written answer. Your paper solution must contain:

1. The circuit (either as a gate sequence or a circuit diagram).
2. The final state $|\Psi(x)\rangle$ obtained after the two gates.
3. The four Z-basis probabilities

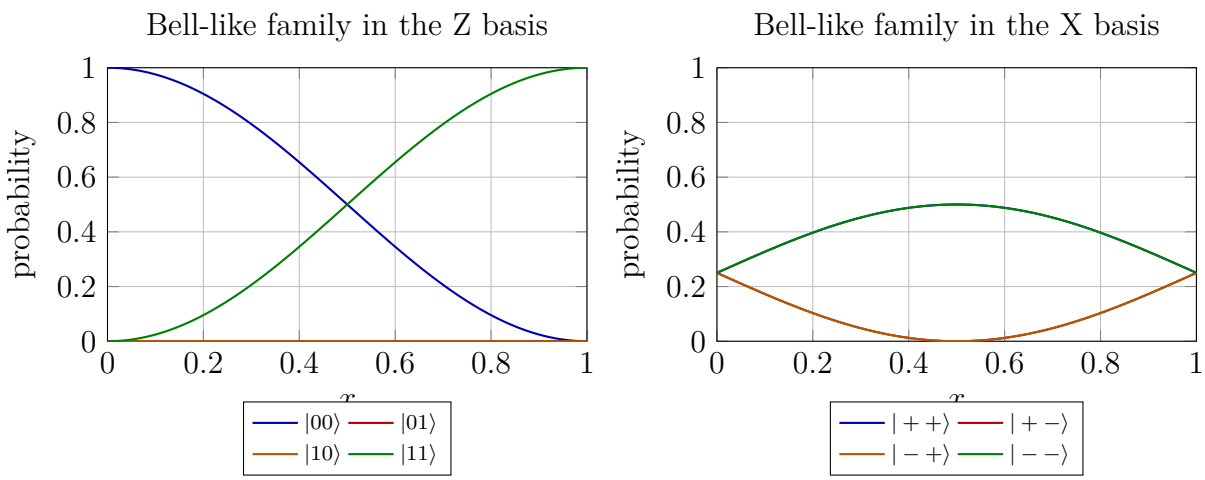
   $$P_Z = (P_{00}, P_{01}, P_{10}, P_{11})$$

   written explicitly in the stated order.
4. The four X-basis probabilities

   $$P_X = (P_{++}, P_{+-}, P_{-+}, P_{--})$$

   written explicitly in the stated order.
5. A short explanation (2–4 sentences):
   - what the Z-basis plot shows about the state,

- what additional information is revealed by the X-basis plot.



Q2.d (coding + discussion). Run a noisy simulation for the same Bell-like circuit from Q2.c:

$$|00\rangle \xrightarrow{R_Y(\pi x) \text{ on } q_0} \xrightarrow{CX(0,1)} .$$

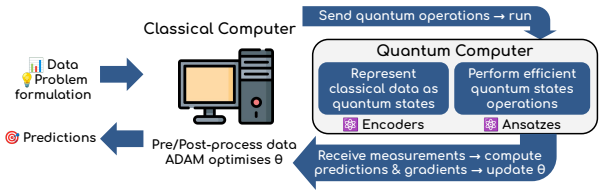
Work in the computational basis and implement `noisy_basis_probabilities()`.

Compare the ideal reference files from Q2.c to the noisy artifacts generated by the tests: in `output/`, see `Q2_d_bell_family_noise_p00.png` and `Q2_d_bell_family_noise_total_variation.png`. In your discussion, comment on at least two visible effects of noise during realistic circuit execution on quantum hardware for this circuit family.

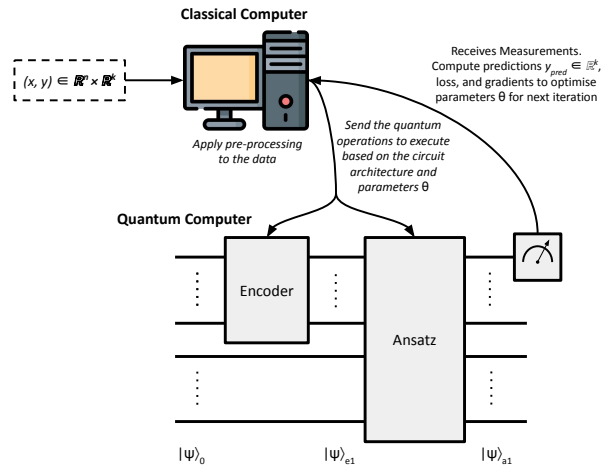
Q3 - Supervised QNN classification with re-uploading (25 pts)

Goal. In this question you will train several variational quantum classifiers on a real binary dataset under noiseless simulation, then revisit one circuit under finite-shot noisy simulation to compare optimizers. You will use cross-entropy loss for classification.

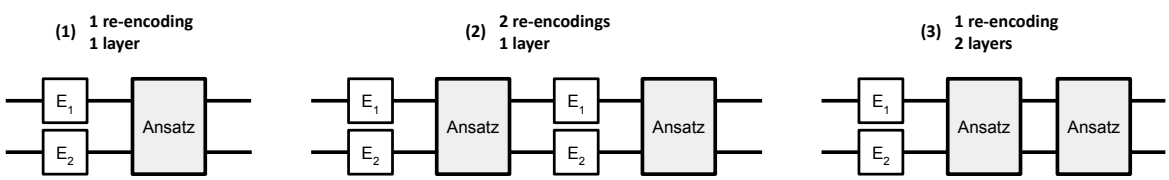
Presentation of Quantum Neural Networks. A Quantum Neural Network (QNN) is a type of Variational Quantum Circuit (VQC) (also called parameterized quantum circuit) that often uses a data re-uploading structure. As presented in Figure **??** and Figure **??**, variational circuit and more precisely QNNs are trained in an hybrid quantum-classical loop where a parameterized quantum circuit is executed on a quantum device or simulator, and a classical optimizer updates the parameters based on the measurement outcomes to minimize a loss function. The structure of the quantum circuit in a QNN often follows a repeated encoder-ansatz pattern, called re-uploading, as illustrated in Figure **??**. More quantum operations and trainable parameters can also be included in the circuits by increasing the number of ansatz layers at each re-uploading step, or by adding more qubits and entangling gates to create more complex architectures.



(a) Variational workflow: trainable quantum circuit inside a classical optimization loop.



(b) Generic QNN pipeline: classical features, quantum processing, and predictions.



(c) Examples of QNN architectures based on encoderansatz repetitions.

Figure 1: Overview of variational quantum circuits and quantum neural network structures.

Dataset and evaluation protocol. Use *sklearn*'s breast-cancer dataset. It is a real binary medical dataset with **30 continuous input features** and **two classes**. To keep the quantum simulation small, we will draw a small stratified subset to keep the same class balance as the original dataset, split it into **train**, **validation**, and **test** sets, and reduce the features with Principal Component Analysis (PCA) in order to convert the original 30 features into a smaller number of informative features that can be encoded into fewer qubits.

The **validation** split is used to compare model families and detect overfitting. The **test** split must stay locked away until the best model family has been chosen from the validation results in order to not leak information from the test set into the model selection process and not bias our final annoucement of performance on unseen data.

Q3.a (paper + coding). Briefly present the dataset and the preprocessing pipeline. In your written answer, give a short description of the dataset, explain the role of the train / validation / test splits, say why PCA is helpful here.

Then implement `load_breast_cancer_pca_splits()` in `src/Q3_vqc_from_scratch.py`.

Q3.b (coding). Build the first classifier family entirely by hand: write your own re-uploading encoder from rotation gates and CNOTs, write your own variational ansatz from rotation gates and CNOTs, organize the circuit as repeated *encoder-ansatz* blocks, and implement a manual training loop with cross-entropy. Implement `build_manual_reuploadi`

`ng_qnn()`, `train_qnn_with_cross_entropy()`, and `predict_from_qnn()`. The Q3 tests will generate training-curve plots such as `output/Q3_b_manual_training_curves.png`.

Q3.c (paper + coding). Build a second manual classifier family in which the encoder also contains trainable parameters. Implement `build_polynomial_reuploading_qnn()` so that it takes integer argument `re_uploading` and each encoder block first maps each retained PCA feature to a trainable polynomial of degree `degree` before using the resulting angles as input of the rotation gates. Use one qubit per retained PCA component, controlled by `n_qubits`. The encoder and ansatz structure should still follow the same repeated block pattern as in Q3.b, but now each encoder block learns its own polynomial map from the classical features to the rotation angles, which gives the model more expressive power before the ansatz acts.

Then compare its validation accuracy to the direct re-uploading model from Q3.b.

Q3.d (coding + discussion). Build a third classifier family from prebuilt Qiskit blocks by implementing `build_prebuilt_qiskit_qnn()`. Then implement `run_q3_model_selection()` so that the direct manual model, the polynomial manual model, and the prebuilt model are all trained on the training split, compared on the validation split only, used to select a single best model family from those validation scores, and only that selected model is finally evaluated on the locked test split. Use the same `re_uploading` value for all three families when performing this comparison. The helper `run_q3_model_selection()` should also take `pca_components` and pass that value both to PCA and to the QNN builders so the circuit width matches the retained feature dimension. With the default `pca_components = 2` and small noiseless QNNs, a locked-test accuracy around `0.75-0.85` is a reasonable target.

The tests will write, in `output/`, `Q3_b_manual_training_curves.png`, `Q3_c_polynomial_training_curves.png`, `Q3_d_prebuilt_training_curves.png`, `Q3_d_best_model_test_predictions.png`, and `Q3_d_model_selection_summary.json`. The test-prediction figure includes the learned decision regions over the 2D PCA plane.

A separate longer test that should run with `pixi run -e qiskit tests_q3_reuploading_sweep`, reruns the validation-only comparison with fewer datapoints for `re_uploading = 1, 2, 3, 4` and writes `Q3_d_reuploading_validation_sweep.png` together with `Q3_d_reuploading_validation_sweep.json`.

Discuss the observed trend in validation accuracy as a function of the number of re-uploading blocks for each model family.

Q3.e (coding + discussion). Separately from the previous comparison, we will use the PyTorch integration with Qiskit to build a hybrid model in which a re-uploading quantum layer is followed by a small classical neural network that serves as a decoder. Implement `train_torch_hybrid_classifier()` and explain why this works with ordinary PyTorch back-propagation in noiseless simulation.

The tests will use this to write `output/Q3_e_torch_hybrid_training_curves.png`, `output/Q3_e_torch_hybrid_test_predictions.png`, and `output/Q3_e_torch_hybrid_summary.json`. The test-prediction figure includes the learned decision regions over the 2D PCA plane.

Discuss the observed training and validation curves, and the final test accuracy. Describe what is being trained in this hybrid model and how it could be critized from the perspective

of quantum machine learning research.

Q3.f (coding + discussion). Reuse one small direct re-uploading classifier from Q3.b, but now evaluate it with finite shots on a noisy simulator instead of noiseless expectation values. Implement `compare_noisy_optimizers()` in `src/Q3_vqc_from_scratch.py` to compare `Adam`, `COBYLA`, and `SPSA` on the same fixed circuit. As in Q3.d and Q3.e, let this function also take `pca_components` so the PCA reduction dimension and the circuit width stay aligned. The file already provides `build_simple_noise_model()` for this noisy part, so use that helper directly rather than designing your own depolarizing noise model. For a fair comparison, all three optimizers must start from the exact same initial weight vector, and your recorded histories should include that shared initial train/validation point before any optimizer update. Keep the train / validation / test discipline from the rest of Q3: compare optimizers with validation accuracy, then report the locked-test accuracy of the optimizer selected from validation.

The tests will write, in `output/`, `Q3_f_noisy_optimizer_training_curves.png`, `Q3_f_noisy_optimizer_test_predictions.png`, and `Q3_f_noisy_optimizer_summary.json`. The test-prediction figure includes a coarse decision-region background over the 2D PCA plane.

In your written answer, discuss the observed training curves and final test accuracies of the three optimizers. How does noise affect the optimization landscape and the stability of different optimizers?

Q4 - sQUlearn abstractions on real datasets (25 pts)

Goal. This question uses two small real datasets from `sklearn` and keeps the same discipline as Q3: split into **train / validation / test**, compare models only with the validation split, and keep the test split locked until one final model has been selected.

Q4.a (paper). Explain clearly the role of the train, validation, and test splits in this question. In particular, state why the test split must stay locked while you compare kernels, regularization values, or circuit depths. What does the test split represent in the context of machine learning research?

Q4.b (coding). Implement `load_wine_pca_splits()` in `src/Q4_squlearn_models.py`. Use the `sklearn` wine dataset: it has 3 classes and 13 chemistry features per sample; keep only `n_samples` stratified samples, split them into train / validation / test, fit `StandardScaler()` on the training split, reduce to 2 principal components with `PCA()`, and rescale those retained components to $[-0.95, 0.95]$.

Q4.c (coding). Implement `train_qsvc_models()` on the wine splits. Use a common encoding circuit and compare `FidelityKernel()` and `ProjectedQuantumKernel()`. For each kernel family, test the regularization values

$$C \in \{0.25, 1.0, 4.0\}.$$

Use validation accuracy to select the single best (kernel, C) pair, then refit only that selected pair on train+validation and evaluate once on test. The tests will generate, in `output/`, `Q4_c_kernel_accuracy_curves.png`, `Q4_c_kernel_test_predictions.png`, and `Q4_c_kernel_s`

`ummary.json`. The test-prediction figure includes the selected kernel model's decision regions over the 2D PCA plane.

Q4.d (coding). Implement `load_linnerud_splits()`. Use the real multi-output `sklearn` Linnerud dataset, keep the regression targets `Weight` and `Waist`, and preprocess the three exercise inputs with standardization, PCA to `n_components` retained features, and a final rescaling to $[-0.95, 0.95]$. This lets Q4.e reuse the first retained PCA features as one feature per qubit. Create train / validation / test splits, standardize the two targets, and return the PCA-reduced inputs.

Q4.e (paper + coding). Implement `build_manual_reuploading_encoding_circuit()` and `train_shared_readout_qnn()`.

The tests will use your returned dictionary to generate, in `output/`, `Q4_e_shared_qnn_layer_mae.png`, `Q4_e_shared_qnn_training_curves.png`, `Q4_e_shared_qnn_feature_comparison.png`, `Q4_e_shared_qnn_circuit_2qubits.png`, `Q4_e_shared_qnn_circuit_3qubits.png`, `Q4_e_shared_qnn_test_predictions.png`, and `Q4_e_shared_qnn_summary.json`, plus `Q4_e_shared_qnn_feature_comparison.json`.

On paper, explain your observations for different numbers of features used and different numbers of re-uploading blocks. Is your model able to learn the two regression targets with the shared-readout approach? Discuss whether "reading multiple outputs from a single quantum state" is a good idea for independent regression targets.

Q4.f (paper). Suppose the deeper QNN gives lower training MAE but worse validation MAE than the shallower QNN. What does that indicate, and give at least two regularization or mitigation techniques you could try.